The Ark AI

Lightning-Powered AI Services Platform

Software Documentation for Copyright Registration

Copyright Work Title:	The Ark AI - Lightning-Powered AI Services Platform
Software Version:	1.0.0
Software Developers:	The Navigator
Software Owners:	The Navigator
Platform URL:	https://arknode.ai
Date:	March 2026

Abstract

The Ark AI is a web-based artificial intelligence services platform that enables users to access over 120 AI-powered task types through a pay-per-task model using Bitcoin Lightning Network micropayments. The platform provides services including text summarization, code generation, translation, research, document analysis, image generation, voice synthesis (including African languages such as Swahili and Amharic), file conversion across 170+ format combinations, and many more professional AI tools.

The platform is built using a proprietary software architecture combining modern web technologies, large language model integration, and Bitcoin Lightning payment infrastructure. It runs as a cloud-hosted web application accessible through any modern web browser on desktop and mobile devices.

1.0 Introduction

The Ark AI is an innovative AI services platform designed to democratize access to artificial intelligence by making it affordable through Bitcoin Lightning micropayments. Unlike traditional AI platforms that require monthly subscriptions or credit card payments, The Ark AI allows users to pay small amounts per task, making AI accessible to anyone with a Lightning wallet.

Key functionalities of the platform include:

- **AI Chat Interface --** Users interact with the AI through a conversational chat interface with conversation memory, supporting text input, voice input, and image uploads.
- **Task Categories --** Over 120 task types organized across categories: coding, writing, research, legal, medical, education, finance, real estate, marketing, document processing, and more.
- **File Converter --** A dedicated file conversion service supporting 170+ conversion types across 5 categories (documents, images, audio, video, spreadsheets) with instant processing.
- **Image Generation --** AI-powered image creation accessible through the chat interface.
- **Voice Services --** Text-to-speech in multiple languages including African languages (Swahili, Amharic), plus audio transcription.
- **Lightning Payments --** Instant micropayments via Bitcoin Lightning Network with QR code invoices.
- **Free Trial System --** New users receive free prompts to try the service before paying.
- **Wallet System --** Users can create in-app wallets, deposit funds via Lightning, and pay for tasks from their balance.
- **Developer API --** Developers can integrate The Ark AI into their applications using a standard API format.
- **PDF Document Generation --** Automatic generation of downloadable PDF documents for applicable task responses.

The platform is designed to be mobile-first, with a responsive interface that works seamlessly on smartphones, tablets, and desktop browsers. All AI processing occurs server-side, requiring no special hardware or software from the user.

2.0 Description of The Ark AI Platform

2.1 Accessing The Ark AI

The Ark AI platform is accessible through the URL https://arknode.ai. The platform loads directly in the browser with no account creation required. Users are presented with a clean, modern chat interface with quick-action buttons for popular tasks. A screenshot of the home page is shown below.

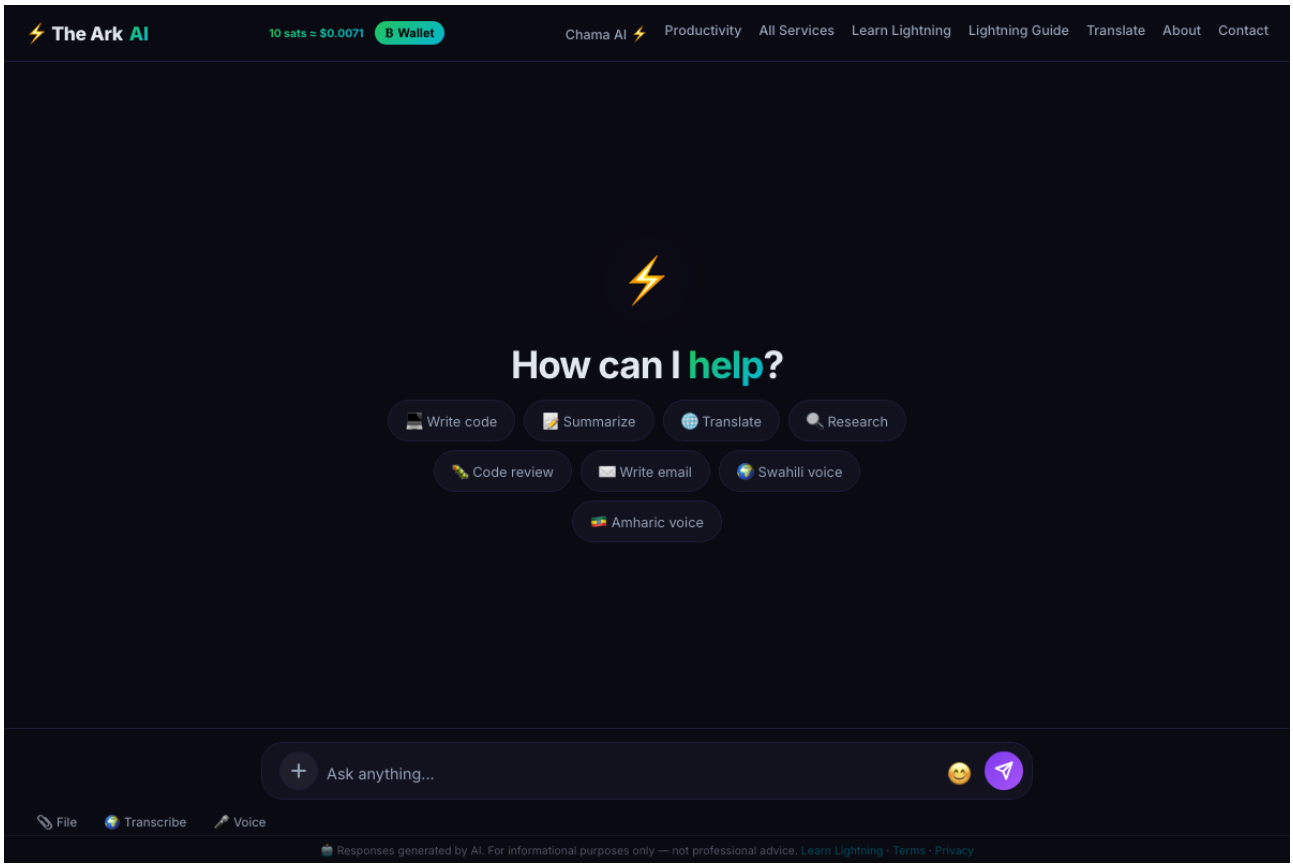

Figure 1: The Ark AI Home Page - Main chat interface with task quick-actions

2.2 Using the AI Chat Interface

Users interact with The Ark AI by typing or speaking their requests into the chat input field. The interface supports text input via keyboard, voice input via microphone, image uploads for visual analysis tasks, and quick-action buttons for common tasks (Write code, Summarize, Translate, Research, etc.). The AI streams responses in real-time and maintains conversation context across messages within the same session.

2.3 Lightning Payment System

When a user submits a task after the free trial, the system generates a Bitcoin Lightning invoice displayed as a QR code. Users scan this QR code with any Lightning wallet to make an instant payment. Once payment is confirmed, the AI processes the task and streams the response. The system uses a proprietary pricing engine that calculates costs based on task complexity.

2.4 File Converter Service

The Ark AI includes a comprehensive file conversion service. This service supports 170+ conversion types across 5 categories: documents (PDF, DOCX, TXT, HTML, RTF, etc.), images (PNG, JPG, WebP, SVG, etc.), audio (MP3, WAV, FLAC, AAC, etc.), video (MP4, AVI, MOV, MKV, etc.), and spreadsheets (CSV, XLSX, ODS, etc.). Users upload a file, select the target format, and receive the converted file instantly with options to download, save to device, or open in a new tab.

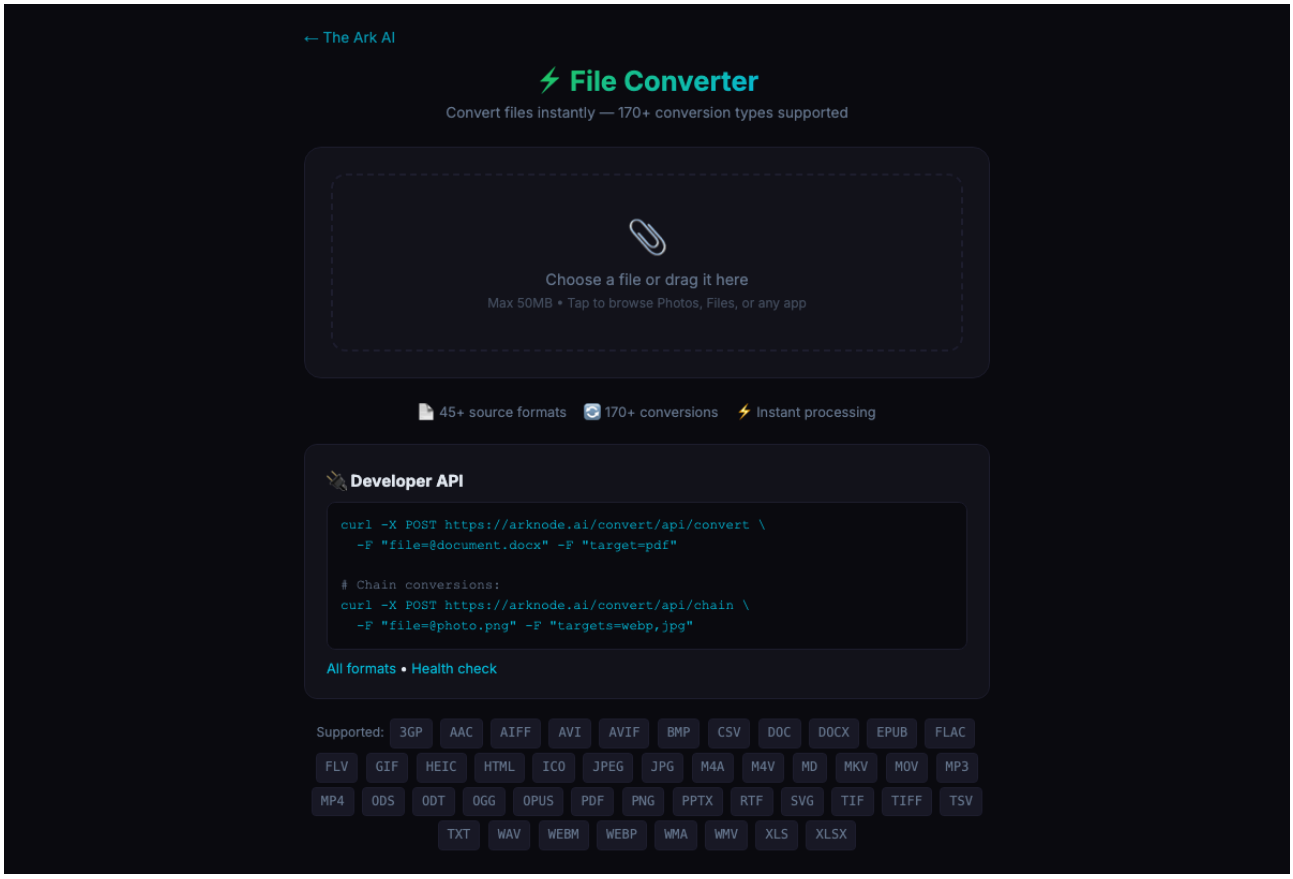

Figure 2: File Converter Interface - Upload and convert files between 170+ formats

2.5 Educational Content

The platform includes an educational section that explains Bitcoin Lightning Network technology, how micropayments work, and how to get started with a Lightning wallet. This serves as user onboarding and educational content about the underlying payment technology.

⚡ The Ark AI

Chat Services Learn Lightning Guide About

⚡ What is Bitcoin Lightning?

A beginner-friendly guide to the payment network that makes AI services instant, cheap, and available to everyone on Earth.

⛈️ Bitcoin Lightning, Simply Explained

💡 The Simple Version

Bitcoin is like a bank vault — super secure, but slow and expensive for small payments. **Lightning** is like a tab at your favorite bar: you open a channel, make instant payments back and forth, and settle up on the main chain later.

🏗️ How It Was Built

Lightning is a **"Layer 2"** network built on top of Bitcoin. Think of Bitcoin as the foundation (Layer 1) and Lightning as the express lane on top. Transactions happen off-chain in payment channels, making them nearly instant and almost free — while still being backed by Bitcoin's security.

~10 min Bitcoin on-chain confirmation	< 1 sec Lightning payment
$0.50–$5+ Bitcoin on-chain fee	< $0.01 Lightning fee

⚙️ How Lightning Payments Work

When you pay with Lightning, your payment hops through a network of channels to reach the recipient — all in under a second.

🔐 It's Trustless & Secure

Lightning uses **cryptographic hash locks** — meaning your payment either fully completes or fully fails. No middlemen can steal your funds. It's peer-to-peer, just like Bitcoin itself, but 1000× faster.

🤖 Why Lightning is Perfect for AI

⚡ Instant

AI responses take seconds. Your payment should too. With Lightning, you pay and get results immediately — no waiting for bank confirmations or card processing.

💰 Micropayments

An AI query might cost 5–50 sats (fractions of a cent). Traditional payments can't handle amounts this small — minimum fees would be more than the service itself. Lightning was *designed* for micropayments.

🌍 Global, No Barriers

No credit card? No bank account? No problem. Lightning works for anyone with a smartphone, anywhere in the world. No KYC, no signup forms, no geographic restrictions.

🔒 Private

No email required. No personal data collected. Pay with sats, get your result, done. Your AI conversations stay between you and the model.

🔑 How The Ark Uses Lightning

The Ark uses the **L402 protocol** (formerly LSAT) — an open standard that combines Lightning payments with API access tokens. Here's how it works behind the scenes:

💬 **1. You send a request**
"Translate this text to French"

🧾 **2. The Ark creates a Lightning invoice**
A small invoice (e.g. 10 sats) with a unique payment hash

3. You pay the invoice

Figure 3: Learn Lightning - Educational page explaining Bitcoin Lightning payments

2.6 In-App Wallet System

Users can create an in-app wallet to maintain a balance for paying for tasks. The wallet allows deposits via Lightning and automatically deducts payment for each task. This provides a seamless experience without needing to create a new Lightning invoice for each request.

2.7 Developer API

The Ark AI provides a developer-facing API that allows third-party applications to integrate AI capabilities. The API supports all task types, streaming responses, and Lightning payment authentication. A dedicated file conversion API is also available for programmatic conversions.

3.0 Software Source Code

The following sections present screenshots of key portions of the source code for The Ark AI platform. The software is a proprietary codebase developed from scratch. Source code screenshots are provided as evidence of original authorship.

3.1 Main Application Server - Core Initialization

```python
"""
Bitcoin AI Agent Server
FastAPI server with L402 Lightning payment gating.

Usage:
    python server.py                    # uses config.yaml
    LNBITS_URL=... python server.py     # env overrides
"""

import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "../data"))
from tracking import track_event, get_report
import sys
import time
import json
import hmac
import hashlib
import base64
import logging
import secrets
import sqlite3
import threading
from pathlib import Path
from typing import Any, Optional

import httpx
import yaml
from dotenv import load_dotenv
from fastapi import FastAPI, Request, Response, HTTPException
from fastapi.responses import JSONResponse
from fastapi.responses import FileResponse as _FileResp
from pydantic import BaseModel
from pdf_gen import generate_pdf, PDF_ELIGIBLE_TASKS, cleanup_old_pdfs, PDF_DIR

from workflows import WORKFLOWS
MAX_INPUT_LENGTH = 50000  # 50KB max input
MAX_TASK_INPUT = 100000  # 100KB for research tasks

from llm_tasks import execute_task as _execute_task_raw, estimate_price, TASK_TYPES, text_to_speech,
generate_image, edit_image, african_text_to_speech, african_transcribe, AFRICAN_TTS_VOICES,
AFRICAN_TTS_LANGUAGES, image_variation, image_upscale, image_bg_remove, image_colorize, image_restore,
image_style_transfer
from llm_tasks import get_model_for_task, FAST_MODEL, POWER_MODEL, POWER_TASKS
from openai_compat import register_openai_routes
from wallet import register_wallet_routes, wallet_deduct, wallet_balance

def _auto_refund(client_ip, amount_sats, payment_hash, reason="task_failure"):
    """Auto-credit a user when their paid task fails."""
    import json, time
    credits_file = "/opt/bitcoin-agent/data/credits.json"
    try:
        try:
            with open(credits_file, "r") as f:
                credits = json.load(f)
        except FileNotFoundError:
            credits = {}
        if client_ip in credits:
            credits[client_ip]["credit_sats"] += amount_sats
        else:
            credits[client_ip] = {
                "credit_sats": amount_sats,
                "reason": reason,
```

Figure 4: server.py - Application initialization, imports, and core configuration

3.2 Task Processing Engine

```python
@app.post("/task/stream")
async def stream_task(req: TaskRequest, request: Request):
    """Execute a task with streaming response (SSE)."""
    client_ip = request.headers.get("X-Real-IP", request.client.host)

    # Check free prompts
    import hashlib as _hl2
    ua = request.headers.get("user-agent", "")
    al = request.headers.get("accept-language", "")
    fp = "fp:" + _hl2.sha256((ua + "|" + al).encode()).hexdigest()[:16]

    is_free = _check_free_prompt_available(client_ip, req.task) and _check_free_prompt_available(fp, req.task)

    # Check wallet
    wallet_id = (req.params or {}).get("wallet_id") or request.headers.get("x-wallet-id")
    has_wallet = False
    price = 0
    if not is_free:
        # Check credits
        _credits_file = "/opt/bitcoin-agent/data/credits.json"
        has_credit = False
        try:
            import json as _cjson
            with open(_credits_file, "r") as _cf:
                _credits = _cjson.load(_cf)
            pricing_c = CONFIG.get("pricing", {})
            price_c = estimate_price(req.task, req.input, pricing_c)
            if client_ip in _credits and _credits[client_ip].get("credit_sats", 0) >= price_c:
                has_credit = True
                price = price_c
        except:
            pass

        if not has_credit:
            pricing = CONFIG.get("pricing", {})
            price = estimate_price(req.task, req.input, pricing)
```

Figure 5: server.py - Streaming task processing endpoint

3.3 File Converter Service

```
"""
The Ark AI — File Converter API
A universal file conversion service powered by FFmpeg, LibreOffice, and ImageMagick.
"""
import os, uuid, time, asyncio, shutil, json, subprocess, logging
from pathlib import Path
from fastapi import FastAPI, UploadFile, File, Form, HTTPException, Request
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware

# PDF conversion tools
try:
    from pdf2docx import Converter as PDFConverter
    HAS_PDF2DOCX = True
except ImportError:
    HAS_PDF2DOCX = False

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("converter")

app = FastAPI(title="The Ark AI File Converter", version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

UPLOAD_DIR = Path("/opt/file-converter/uploads")
OUTPUT_DIR = Path("/opt/file-converter/converted")
STATIC_DIR = Path("/opt/file-converter/static")
MAX_FILE_SIZE = 50 * 1024 * 1024  # 50MB
FILE_EXPIRY = 3600  # 1 hour

UPLOAD_DIR.mkdir(exist_ok=True)
OUTPUT_DIR.mkdir(exist_ok=True)

# === Supported Conversions ===
CONVERSIONS = {
    # Documents
    "pdf": {"from":
["docx","doc","odt","txt","html","rtf","epub","xlsx","csv","pptx","png","jpg","jpeg","bmp","tiff","gif","webp"],
"tool": "libreoffice"},
    "docx": {"from": ["pdf","odt","txt","html","rtf"], "tool": "libreoffice"},
    "txt": {"from": ["pdf","docx","odt","rtf","html"], "tool": "libreoffice"},
    "html": {"from": ["docx","odt","txt","md"], "tool": "libreoffice"},
    "odt": {"from": ["docx","doc","txt","rtf"], "tool": "libreoffice"},
    # Images
    "png": {"from": ["jpg","jpeg","webp","bmp","tiff","tif","gif","svg","heic","ico","avif"], "tool":
"imagemagick"},
    "jpg": {"from": ["png","webp","bmp","tiff","tif","gif","svg","heic","ico","avif"], "tool": "imagemagick"},
    "jpeg": {"from": ["png","webp","bmp","tiff","tif","gif","svg","heic","ico","avif"], "tool": "imagemagick"},
    "webp": {"from": ["png","jpg","jpeg","bmp","tiff","gif","svg"], "tool": "imagemagick"},
    "gif": {"from": ["png","jpg","jpeg","webp","bmp","mp4","mov","avi"], "tool": "mixed"},
    "bmp": {"from": ["png","jpg","jpeg","webp","tiff"], "tool": "imagemagick"},
    "tiff": {"from": ["png","jpg","jpeg","webp","bmp"], "tool": "imagemagick"},
    "ico": {"from": ["png","jpg","jpeg","svg"], "tool": "imagemagick"},
    "svg": {"from": ["png","jpg","jpeg"], "tool": "imagemagick"},
    # Audio
    "mp3": {"from": ["wav","flac","m4a","ogg","aac","wma","opus","aiff","webm"], "tool": "ffmpeg"},
    "wav": {"from": ["mp3","flac","m4a","ogg","aac","wma","opus","aiff"], "tool": "ffmpeg"},
    "flac": {"from": ["mp3","wav","m4a","ogg","aac","wma"], "tool": "ffmpeg"},
    "m4a": {"from": ["mp3","wav","flac","ogg","aac","wma"], "tool": "ffmpeg"},
    "ogg": {"from": ["mp3","wav","flac","m4a","aac"], "tool": "ffmpeg"},
    "aac": {"from": ["mp3","wav","flac","m4a","ogg"], "tool": "ffmpeg"},
    # Video
    "mp4": {"from": ["avi","mov","mkv","webm","flv","wmv","m4v","3gp","gif"], "tool": "ffmpeg"},
```

Figure 6: file-converter/server.py - File conversion engine

4.0 Declaration of Originality

I hereby declare that The Ark AI software described in this documentation is an original work developed by the undersigned. The software architecture, user interface design, payment integration logic, task processing engine, and all associated source code were created independently and do not infringe on the intellectual property rights of any third party.

The software integrates with third-party APIs and open-source tools (as is standard practice in modern software development), but the integration logic, user experience design, business logic, and overall system architecture are original proprietary works.

Signature: _____

Name: The Navigator

Date: March 2026